Value-Based Methods: Q-Learning to DQN

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

KAUST Academy
King Abdullah University of Science and Technology

May 6, 2026

Table of Contents

Learning Outcomes

By the end of this session, you will be able to:

- Understand and implement **Deep Q-Networks (DQN)**.
- Grasp the **SARSA algorithm** and how it contrasts with Q-learning.
- Analyze the **differences between off-policy and on-policy learning**.
- Understand **Experience Replay** and **Target Networks** as stabilisation techniques.
- Understand **Double DQN** and the problem of overestimation bias.
- Describe the key extensions in the **DQN family**: Dueling, Prioritized Replay, Distributional RL, and Rainbow.

Where We Left Off: From Day 1 to Day 2

Day 1 Problem	**Day 2 Solution**
Curse of dimensionality	Neural network function approximator
Correlated samples $\Rightarrow$ instability	Experience Replay
Moving target $\Rightarrow$ divergence	Target Network
Overestimation bias	Double DQN

Reinforcement Learning: **SARSA**

- **SARSA** stands for:

State
$\rightarrow$ Action
$\rightarrow$ Reward
$\rightarrow$ State$'$
$\rightarrow$ Action$'$

- **On-policy learning:** Learns Q-values by following the current behavior policy.
- **Update Rule:**

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

SARSA: Key Points

- Uses the action the agent actually takes, not just the best possible one.
- Learns from the agent's real experience, not hypothetical alternatives.
- Tends to be safer—useful in situations where mistakes are costly.
- Helps the agent improve its actual behavior step by step.

SARSA vs. Q-Learning

- The SARSA algorithm is a slight variation of the Q-Learning algorithm.
- Q-Learning is **off-policy**: the Bellman update uses $\max_{a'} Q(s', a')$, which asks "what is the *best possible* action in s'?"—regardless of what the agent actually does.
- SARSA is **on-policy**: the update uses $Q(s', a')$ where a' is the action the current policy actually selects, so it learns from real experience, not hypothetical alternatives.

SARSA vs. Q-Learning

- The SARSA algorithm is a slight variation of the Q-Learning algorithm.
- Q-Learning is **off-policy**: the Bellman update uses $\max_{a'} Q(s', a')$, which asks "what is the *best possible* action in s'?"—regardless of what the agent actually does.
- SARSA is **on-policy**: the update uses $Q(s', a')$ where a' is the action the current policy actually selects, so it learns from real experience, not hypothetical alternatives.

Q-Learning: $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

SARSA vs. Q-Learning

- The SARSA algorithm is a slight variation of the Q-Learning algorithm.
- Q-Learning is **off-policy**: the Bellman update uses $\max_{a'} Q(s', a')$, which asks "what is the *best possible* action in s'?"—regardless of what the agent actually does.
- SARSA is **on-policy**: the update uses $Q(s', a')$ where a' is the action the current policy actually selects, so it learns from real experience, not hypothetical alternatives.

Q-Learning: $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r(s_t, a_t) + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

SARSA: $$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r(s_t, a_t) + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right]$$

- The update equation for SARSA depends on the current state, current action, reward obtained, next state, and next action.
- This observation led to the naming of the learning technique as **SARSA**, which stands for **State-Action-Reward-State-Action**, symbolizing the tuple (s, a, r, s', a').

SARSA

- The update equation for SARSA depends on the current state, current action, reward obtained, next state, and next action.
- This observation led to the naming of the learning technique as **SARSA**, which stands for **State-Action-Reward-State-Action**, symbolizing the tuple (s, a, r, s', a').
- Similar to DQN, there is also a Deep SARSA variant.

SARSA Algorithm

Input : States, $s \in S$, Actions $a \in A(s)$, Initialize Q(s, a), α, γ, π to an arbitrary policy (non-greedy)
Output: Optimal action value Q(s, a) for each stste-action pair
while *True* do
  for ($i = 0$; $i \leq$ *# of episodes*; $i++$) do
    Initialize s
    Choose a from s, using policy derived from Q
    Repeat(for each step of episodes):
    Take action a; observe reward, r, and next state, s'
    Choose action a' from state s' using policy derived from Q
    $Q(s, a) \longleftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
    $s \longleftarrow s'$; $a \longleftarrow a'$;
    until s is terminal
  end
end

[0]Lockery & Peters, Adaptive learning by a target-tracking system

Q-Learning vs SARSA: Comparison Table

Feature	Q-Learning	SARSA
Policy Type	Off-policy	On-policy
Learns From	Others' or past experience	Own current experience
Target	$\max Q(s', a')$	$Q(s', a')$ actually taken
Exploration Aware	No	Yes
Risk Sensitivity	More aggressive	More conservative
Use Case	Goal-seeking behavior	Risk-aware behavior

Reinforcement Learning: **Training the Q-network**

Training the Q-network: Experience Replay

- Learning from batches of consecutive samples is problematic:
  - Samples are correlated $\Rightarrow$ inefficient learning.
  - The current Q-network parameters determine the next training samples (e.g., if the maximizing action is to move left, training samples will be dominated by transitions from the left-hand side), which can lead to bad feedback loops.

Training the Q-network: Experience Replay

- Learning from batches of consecutive samples is problematic:
  - Samples are correlated $\Rightarrow$ inefficient learning.
  - The current Q-network parameters determine the next training samples (e.g., if the maximizing action is to move left, training samples will be dominated by transitions from the left-hand side), which can lead to bad feedback loops.
- These problems can be addressed using experience replay:
  - Maintain a replay memory buffer of transitions (s_t, a_t, r_t, s_{t+1}) as episodes are played.
  - Train the Q-network on random minibatches of transitions sampled from the replay memory, instead of using consecutive samples.
- Note: the replay buffer is a general off-policy tool—it is not specific to DQN.

DQN Algorithm

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

[0] ε-greedy exploration—covered in Day 1. ε is annealed from high to low over training.

DQN with Target Network

- DQN, as stated above, will have learning problems because the target and the prediction are not independant as they both rely on the same network.
- It's like a dog chasing it's own tail.

DQN with Target Network

- DQN, as stated above, will have learning problems because the target and the prediction are not independant as they both rely on the same network.
- It's like a dog chasing it's own tail.
- **Solution:** Use two separate Q-value estimators, each of which is used to update the other.
- The target values are calculated using a target Q-network. The target Q-network's parameters are updated to the current networks every C time steps.
- Target network prevents the network from spiraling around.

DQN with Target Network Algorithm

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^{-} = \theta$

For episode $= 1, M$ **do**

  Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

  For $t = 1$,T **do**

    With probability ε select a random action a_t

    otherwise select $a_t = \text{argmax}_a Q(\phi(s_t), a; \theta)$

    Execute action a_t in emulator and observe reward r_t and image x_{t+1}

    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

    Store transition $\left(\phi_t, a_t, r_t, \phi_{t+1}\right)$ in D

    Sample random minibatch of transitions $\left(\phi_j, a_j, r_j, \phi_{j+1}\right)$ from D

    Set $y_j = \begin{cases} r_j & \text{if episode terminates at step j+1} \\ r_j + \gamma \max_{a'} \hat{Q}\left(\phi_{j+1}, a'; \theta^{-}\right) & \text{otherwise} \end{cases}$

    Perform a gradient descent step on $\left(y_j - Q\left(\phi_j, a_j; \theta\right)\right)^2$ with respect to the network parameters θ

    Every C steps reset $\hat{Q} = Q$

  End For

End For

[0]Minh et al. https://www.nature.com/articles/nature14236

Reinforcement Learning: **Double DQN**

Double DQN

The problem. Even with a target network, the target computation still has a flaw:

$$y = r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)$$

The target network both *selects* the best next action (via max) and *scores* it (via $\hat{Q}$) in one step. Taking a max over noisy estimates will almost always pick an overestimated action—and then that overestimate becomes the training target.

Double DQN

The problem. Even with a target network, the target computation still has a flaw:

$$y = r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)$$

The target network both *selects* the best next action (via max) and *scores* it (via $\hat{Q}$) in one step. Taking a max over noisy estimates will almost always pick an overestimated action—and then that overestimate becomes the training target.

The fix. Double DQN splits the job between the two networks we already have:

$$y = r + \gamma \, \hat{Q}\Big(s', \ \arg\max_{a'} Q(s', a'; \theta); \ \theta^-\Big)$$

The **online network** θ picks which action looks best. The **target network** θ^- scores that action. Because their weights differ, their errors do not overlap—overestimation bias is reduced. This is a one-line change to the DQN code.

Target Network vs. Double DQN

Both use two networks, which makes them easy to confuse. They solve completely different problems:

	Problem solved	**How**
Target Network	Unstable, moving target	Freeze weights for C steps
Double DQN	Overestimation bias	Split action selection from evaluation

The two-network structure was already there from the Target Network. Double DQN just changes *how* those two networks are used.

DDQN Algorithm

Algorithm 1: Double DQN Algorithm.

input : $\mathcal{D}$ – empty replay buffer; θ – initial network parameters, θ^- – copy of θ
input : N_r – replay buffer maximum size; N_b – training batch size; N^- – target network replacement freq.
for *episode* $e \in \{1, 2, \ldots, M\}$ **do**
  Initialize frame sequence $\mathbf{x} \leftarrow ()$
  for $t \in \{0, 1, \ldots\}$ **do**
    Set state $s \leftarrow \mathbf{x}$, sample action $a \sim \pi_{\mathrm{B}}$
    Sample next frame x^t from environment $\mathcal{E}$ given (s, a) and receive reward r, and append x^t to $\mathbf{x}$
    if $|\mathbf{x}| > N_f$ **then** delete oldest frame $x_{t_{\min}}$ from $\mathbf{x}$ **end**
    Set $s' \leftarrow \mathbf{x}$, and add transition tuple (s, a, r, s') to $\mathcal{D}$,
      replacing the oldest tuple if $|\mathcal{D}| \geq N_r$
    Sample a minibatch of N_b tuples $(s, a, r, s') \sim \mathrm{Unif}(\mathcal{D})$
    Construct target values, one for each of the N_b tuples:
    Define $a^{\max}(s'; \theta) = \arg\max_{a'} Q(s', a'; \theta)$

$$y_j = \begin{cases} r & \text{if } s' \text{ is terminal} \\ r + \gamma Q(s', a^{\max}(s'; \theta); \theta^-), & \text{otherwise.} \end{cases}$$

    Do a gradient descent step with loss $\|y_j - Q(s, a; \theta)\|^2$
    Replace target parameters $\theta^- \leftarrow \theta$ every N^- steps
  end
end

[0] Van Hasselt, Guez & Silver, *Deep Reinforcement Learning with Double Q-learning*, AAAI 2016

Case Study: **Playing Atari Games**

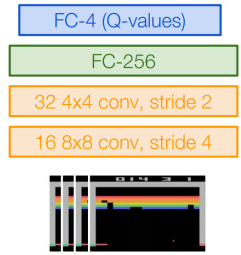
Case Study: Playing Atari Games

- **Objective**: Complete the game with the highest score
- **State**: Raw pixel inputs of the game state
- **Action**: Game controls e.g. Left, Right, Up, Down
- **Reward**: Score increase/decrease at each time step

[0][Mnih et al. NIPS Workshop 2013; Nature 2015]

Case Study: Playing Atari Games

$Q(s, a; \theta)$:
neural network
with weights θ



Last FC layer has 4-d output (if 4 actions), corresponding to $Q(s_t, a_1)$, $Q(s_t, a_2)$, $Q(s_t, a_3)$, $Q(s_t, a_4)$

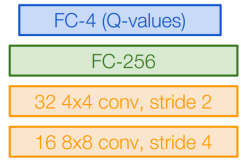
Number of actions between 4-18 depending on Atari game

Current state s_t: 84x84x4 stack of last 4 frames
(after RGB->grayscale conversion, downsampling, and cropping)

[0][Mnih et al. NIPS Workshop 2013; Nature 2015]

Case Study: Playing Atari Games

$Q(s, a; \theta)$: neural network with weights θ



Last FC layer has 4-d output (if 4 actions), corresponding to $Q(s_t, a_1)$, $Q(s_t, a_2)$, $Q(s_t, a_3)$, $Q(s_t, a_4)$

Number of actions between 4-18 depending on Atari game

Current state s_t: 84x84x4 stack of last 4 frames
(after RGB->grayscale conversion, downsampling, and cropping)

https://www.youtube.com/watch?v=V1eYniJ0Rnk

[0][Mnih et al. NIPS Workshop 2013; Nature 2015]

DQN Family & SARSA: **Summary**

When to Use Which Algorithm?

Scenario	Recommended Algorithm
Environment is stochastic	SARSA (conservative)
Maximizing reward is the priority	Q-Learning or DQN
Large state space (e.g., images)	DQN
Limited computational resources	SARSA (simpler model)

Future Directions

DQN works well but has known weaknesses. Each extension below addresses one specific problem. Together they form the DQN family.

- **Dueling DQN**: Splits Q into $V(s)$ and $A(s, a)$—learns which states are valuable without evaluating every action.
- **Prioritized Replay**: Samples transitions proportional to TD error instead of uniformly—prioritises what the network got wrong.
- **Distributional RL**: Learns the full return distribution $Z(s, a)$, not just the expected value—richer signal, faster learning.
- **Noisy Nets**: Replaces ε-greedy with learned noise in the weights—the network decides how much to explore.
- **Rainbow**: Combines all of the above. Represents the practical ceiling of value-based methods.

Summary

- SARSA is an on-policy alternative to Q-learning, using the action the agent actually takes rather than $\max_{a'} Q(s', a')$.
- The on-policy vs. off-policy distinction is one of the most important axes in RL algorithm design.
- Deep Q-Learning uses a neural network as a function approximator for Q-values, enabling RL on high-dimensional inputs.
- Experience replay breaks correlation between consecutive samples; the target network stabilizes the moving target.
- Double DQN decouples action selection from evaluation to reduce overestimation bias.

Summary (cont.)

- The DQN family extensions (Dueling, Prioritized Replay, Distributional, Noisy Nets, Rainbow) each address a specific weakness of vanilla DQN.

DQN Family & SARSA: **References**

References

[1] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015).
Human-level control through deep reinforcement learning.
Nature.

[2] Van Hasselt, H., Guez, A., Silver, D. (2016).
Deep Reinforcement Learning with Double Q-learning.
AAAI.

[3] Wang, Z., Schaul, T., Hessel, M., et al. (2016).
Dueling Network Architectures for Deep RL.
ICML.

References (cont.)

[4] Schaul, T., Quan, J., Antonoglou, I., Silver, D. (2016).
Prioritized Experience Replay.
ICLR.

[5] Hessel, M., Modayil, J., Van Hasselt, H., et al. (2018).
Rainbow: Combining Improvements in Deep Reinforcement Learning.
AAAI.

[6] Bellemare, M. G., Dabney, W., Munos, R. (2017).
A Distributional Perspective on Reinforcement Learning.
ICML.

References (cont.)

[7] Rummery, G. A., & Niranjan, M. (1994).
On-line Q-learning using connectionist systems.
Technical Report.

[8] Sutton, R. S., & Barto, A. G. (2018).
Reinforcement Learning: An Introduction.

[9] Berkeley CS285 Deep RL Lectures.
https://rail.eecs.berkeley.edu/deeprlcourse/

[10] David Silver's RL Series.
http://www0.cs.ucl.ac.uk/staff/d.silver/web/Teaching.html

[11] Emma Brunskill, Stanford CS234: Reinforcement Learning.
https://web.stanford.edu/class/cs234/

References (cont.)

[12] Fei-Fei Li, Yunzhu Li & Ruohan Gao, Stanford CS231n: Deep Learning for Computer Vision.
http://cs231n.stanford.edu/

[13] Ashwin Rao, Stanford CME241: Foundations of Reinforcement Learning with Applications in Finance.
https://web.stanford.edu/class/cme241/

[14] Jimmy Ba & Bo Wang, UofT CSC413/2516: Neural Networks and Deep Learning.
https://uoft-csc413.github.io/2022/

Credits

Dr. Prashant Aparajeya

Computer Vision Scientist — Director(AISimply Ltd)

p.aparajeya@aisimply.uk

This project benefited from external collaboration, and we acknowledge their contribution with gratitude.